Streamlit 소개 및 활용 가이드

1. Streamlit 개요

- Streamlit은 Python 기반의 오픈소스 웹 애플리케이션 프레임워크로, 데이터 사이언스 및 머신러닝 모델을 빠르게 배포하고 공유할 수 있도록 설계된 도구
  - Flask, Django 같은 전통적인 웹 프레임워크보다 훨씬 간결한 코드로 대시보드 구축 가능
  - 머신러닝 모델의 입력과 출력을 쉽게 시각화하고, 웹 인터페이스를 손쉽게 만들 수 있음
- 주요 특징
  - 간편한 구현
  - 백엔드 개발 불필요
  - 자동 리렌더링: UI 요소가 변경되면 자동으로 애플리케이션이 업데이트됨
  - 배포 기능 지원: Streamlit Cloud를 이용해 쉽게 배포 가능
  - 인터랙티브 UI: 사용자가 입력한 데이터를 즉시 반영할 수 있는 위젯 지원
  - 다양한 차트 및 지도 지원: 데이터를 시각적으로 표현할 수 있는 다양한 차트 및 지도 위젯을 기본 제공
  - 커뮤니티 및 확장 기능 지원: 다양한 플러그인과 사용자 정의 컴포넌트를 활용하여 기능 확장 가능
- 실행 방법: 터미널에서 `streamlit run [파일명].py` 명령어를 통해 로컬 서버를 가동

2. 주요 API

2.1 텍스트 출력

함수명	용도 및 특징
`st.write`	문자열, 숫자, 데이터프레임, 리스트, 객체 등 출력 가능한 범용 출력 함수
`st.title`	애플리케이션 최상위 제목 표시 및 메인 섹션 정의. 가장 큰 글씨 크기로 표시

`st.header`	주요 섹션 구분을 위한 대형 제목 출력
`st.subheader`	세부 항목 분류를 위한 중형 제목 출력
`st.markdown`	Markdown 문법을 활용한 텍스트 서식화 및 링크 삽입
`st.caption`	주석, 출처 표시 등 작은 글꼴의 보조 텍스트 출력
`st.divider`	섹션 간 경계를 긋는 수평선 삽입
`st.code`	언어별 구문 강조가 적용된 코드 블록 출력
`st.latex`	수학 공식 및 기호를 렌더링할 때 사용

- Magic 기능
  - 별도의 함수 호출 없이도 변수나 리터럴 값을 화면에 바로 출력할 수 있는 편리한 기능
  - 작성 방식: 별도의 함수 호출 없이 변수명이나 값만 단독으로 작성(ex. `data`)
  - 동작 원리: Streamlit이 내부적으로 자동으로 `st.write()`를 호출함

2.2 사용자 입력 받기

함수명	용도 및 특징
`st.text_input`	사용자가 텍스트를 입력할 수 있는 입력 상자 제공
`st.number_input`	사용자가 숫자를 입력할 수 있는 입력 상자 제공
`st.date_input`	사용자가 날짜를 선택할 수 있는 입력 상자 제공
`st.time_input`	사용자가 시간을 선택할 수 있는 입력 상자 제공
`st.text_area`	사용자가 여러 줄의 텍스트를 입력할 수 있는 입력 필드 제공
`st.file_uploader`	사용자가 파일을 업로드할 수 있도록 함
`st.color_picker`	사용자가 색상을 선택할 수 있도록 함

2.3 버튼 및 선택형 입력

Streamlit은 다양한 버튼 및 선택형 입력 위젯을 제공하여 인터랙티브한 UI를 만들 수 있음

함수명	용도 및 특징

`st.button`	사용자가 버튼을 클릭하면 특정 동작을 실행할 수 있도록 함
`st.checkbox`	체크박스를 추가하여 사용자가 선택할 수 있도록 함
`st.radio`	사용자가 여러 옵션 중 하나를 선택할 수 있도록 함
`st.selectbox`	사용자가 드롭다운 리스트에서 하나의 옵션을 선택할 수 있도록 함
`st.multiselect`	사용자가 드롭다운 리스트에서 여러 개의 옵션을 동시에 선택할 수 있도록 함
`st.slider`	사용자가 슬라이더를 사용해 숫자 값을 조정할 수 있도록 함
`st.select_slider`	사용자가 리스트에서 정해진 값을 선택하는 슬라이더

2.4 데이터 출력

Streamlit을 활용하면 이미지, 비디오, 오디오 파일뿐만 아니라 데이터프레임, JSON, 주요 지표(Metrics) 등을 쉽게 출력 가능

(1) 이미지, 비디오 및 오디오 파일 출력

- `st.image` : 이미지 출력하는 함수로, PIL.Image 객체 또는 이미지 URL, 로컬 파일 경로 사용 가능
- `st.audio` : 오디오 파일을 웹에서 재생할 수 있도록 함
- `st.video` : 비디오 파일을 웹에서 재생할 수 있도록 함

(2) 데이터프레임 출력

- `st.write` : 데이터프레임을 기본적인 형태로 출력 가능
- `st.dataframe` : 스크롤 및 정렬 기능이 포함된 인터랙티브한 데이터프레임 출력 가능
- `st.table` : 정적인 테이블 형태로 데이터 출력, 데이터 변경이 필요하지 않을 때 유용

(3) JSON 데이터 출력

- `st.json` : JSON 형태의 데이터를 계층적으로 정리된 형태로 출력 가능

(4) 주요 지표 및 통계 출력

- `st.metric` : 특정 지표를 강조해서 보여줄 때 사용되며, `delta` 값을 설정하면 변화량 표시 가능

2.5 차트 및 지도 출력

데이터 시각화는 데이터를 더욱 직관적으로 이해하고 인사이트를 얻는 데 중요한 역할을 함. Streamlit을 활용하면 간단한 코드로 다양한 차트와 지도 데이터를 시각화할 수 있음.

(1) 차트 출력

- Matplotlib 그래프 → `st.pyplot`
  - `matplotlib.pyplot`을 사용하여 생성한 그래프를 Streamlit 앱에 표시 가능
  - Matplotlib은 Python에서 가장 널리 사용되는 시각화 라이브러리로, 세밀한 커스터마이징이 필요한 경우 유용함
  - 히스토그램, 산점도, 선 그래프, 막대 그래프 등 다양한 그래프 직접 제어 가능
- 선형 차트 → `st.line_chart`
  - Pandas 데이터프레임을 이용해 선형 차트를 그림
  - 시간에 따른 변화를 시각화할 때 사용
  - 시계열 데이터, 주가 변동, 트렌드 분석 등에 유용함
- 막대 차트 → `st.bar_chart`
  - Pandas 데이터프레임을 이용해 막대 차트를 그림
  - 카테고리별 비교 시 적절한 차트
  - 각 범주의 크기 차이를 강조하는 데 유용함
- 영역 차트 → `st.area_chart`
  - Pandas 데이터프레임을 이용해 영역 차트를 그림
  - 선형 차트와 유사하지만, 누적된 값의 변화를 강조할 때 유용함
  - 전체적인 변화 흐름과 차이를 강조할 때 사용됨
- Altair 차트 → `st.altair_chart`
  - Altair를 사용해 인터랙티브한 차트 생성
  - 대규모 데이터 분석 및 상호작용이 필요한 경우 적합함
  - 산점도, 버블 차트 등 다차원 데이터를 시각화하는 데 유용함
- 그래프 구조 시각화 → `st.graphviz_chart`

- graphviz를 이용해 노드 및 엣지 기반 그래프 생성
- 노드와 엣지 기반 네트워크를 표현하는 데 유용함
- 계층 구조, 프로세스 흐름도, 관계 모델링 등에 사용됨

(2) 지도 출력

- 지도 출력 → `st.map`
  - Pandas 데이터프레임에 위도와 경도 값을 포함하여 지도 출력 가능
  - 지리적 데이터 시각화를 위한 기본적인 지도 기능을 제공
  - 위도와 경도 값이 포함된 데이터가 필요

2.6 상태 메시지 및 진행률 표시

Streamlit은 애플리케이션의 현재 상태를 시각적으로 표현할 수 있는 다양한 기능 제공

(1) 진행률 및 로딩 상태 표시

함수명	용도 및 특징
`st.progress`	프로세스 진행 상황을 나타내는 진행률 바 생성
`st.spinner`	시간이 걸리는 작업 중 로딩 메시지 표시
`st.balloons`	사용자가 작업을 완료했을 때 풍선 애니메이션을 표시해 축하

(2) 상태 메시지 출력

함수명	용도 및 특징
`st.success`	성공적인 작업 완료를 알리는 녹색 메시지 출력
`st.error`	에러 또는 실패한 작업을 알리는 빨간색 메시지 출력
`st.warning`	주의가 필요한 작업을 알리는 노란색 메시지 출력
`st.info`	일반적인 정보나 가이드라인을 전달하는 파란색 메시지 출력
`st.exception`	Python의 예외를 출력

3. Streamlit Component

3.1 Sidebar 사용

- 기본 사용법
  - `st.sidebar`을 사용하면 앱의 주요 콘텐츠를 방해하지 않고 옵션, 입력 필드, 버튼 등을 배치 가능
- 특징: 주요 콘텐츠를 화면 중앙에 배치하면서 설정을 사이드바에 넣어 사용자 경험 향상

3.2 여러 개의 컬럼 활용

- 기본 사용법
  - `st.columns`을 사용하여 여러 개의 컬럼 생성 → 레이아웃 정돈
  - `st.columns(n)`을 사용하면 n개의 동일한 크기의 컬럼 생성
  - `st.columns([비율1, 비율2, …])`을 사용하면 비율을 지정해 가변적 너비의 컬럼 생성
- 특징: 여러 개의 UI 요소를 병렬 배치 가능 → 데이터 시각화 및 비교 분석 화면 구성에 유용함

3.3 컨테이너 사용하기

- 기본 사용법
  - `st.container`을 사용해 내용을 그룹화할 수 있음
  - `st.container` 안에 요소를 배치하면 논리적인 그룹 생성 가능
- 특징: 반복적 UI 요소를 동적으로 생성할 때 유용함

3.4 Tab 활용하기

- 기본 사용법

    - `st.tabs(["탭 이름1", "탭 이름2", ...])`을 사용하여 탭을 생성하고, 각 탭에 콘텐츠 추가 가능
- 특징: 여러 개의 정보를 한 화면에서 관리하면서도 사용자가 선택적으로 볼 수 있도록 구성했으며, 멀티 페이지 구조를 효과적으로 구현 가능

3.5 Expander 사용하기

- 기본 사용법
    - `st.expander(”제목”)`을 사용해 정보를 접고 펼치는 UI 구현 가능
    - 내부에 이미지 및 텍스트 추가 가능
- 특징: 필요할 때만 내용을 볼 수 있도록 구성할 수 있어 설명서나 추가 정보 제공 시 유용함